RELATÓRIO DE SEGURANÇA

Sistema MedFlow

Clínica Odontológica — Auditoria de código e banco de dados

Escopo: Backend Flask/JWT · Banco de dados MySQL · Configuração e segredos · Autenticação
Data da auditoria: 30 de setembro de 2026
Versão do documento: 1.0
Solicitante: Paulo (analista de sistemas / responsável do projeto)

DOCUMENTO CONFIDENCIAL

1. Sumário executivo

Esta auditoria revisou todo o código do backend (Flask + JWT + SQLAlchemy), o esquema e os dados de carga do banco de dados MySQL, o tratamento de configuração e segredos, e a camada de autenticação/autorização do sistema MedFlow. O objetivo foi identificar riscos de segurança antes de qualquer uso com dados reais de pacientes.

A base de código está **bem construída do ponto de vista de segurança de aplicação**: todas as rotas de escrita exigem login e perfil autorizado, as senhas usam bcrypt, as consultas SQL são parametrizadas (sem risco de SQL injection), o upload de arquivos é validado de forma robusta e os segredos vêm de variáveis de ambiente. As correções da revisão anterior (2026-08-17) foram confirmadas como presentes e funcionando.

Os pontos que exigem ação **não estão no código da aplicação, mas na configuração de implantação**: o arquivo `.env` em uso ainda contém valores de exemplo (placeholders) no lugar das chaves secretas reais, e o script de dados de demonstração define uma senha de administrador publicamente conhecida. Esses dois itens precisam ser resolvidos antes de qualquer publicação.

> **Veredito:** Adequado para fins acadêmicos e de demonstração. **Não liberar com pacientes reais** antes de resolver os 2 achados de severidade Alta e passar por um pentest humano (ver Seção 6).

Panorama dos achados

Severidade	Qtde.	Resumo
ALTA	2	Segredos ainda em valor de exemplo; senha de admin de demonstração é pública
MÉDIA	4	Leitura ampla de dados de paciente; rate-limit fragmentado entre workers; sem revogação de token; sem HTTPS imposto pela aplicação
BAIXA	4	Sem MFA para admin; política de senha mínima; sem vínculo profissional-paciente; trava do HTML avulso não é segurança

2. Escopo e metodologia

Arquivos revisados: `backend/app/` (factory, config, extensions, models, routes, utils), `backend/requirements.txt`, `backend/Dockerfile`, `backend/wsgi.py`, `database/schema.sql`, `database/seed_demo.sql`, `.env` / `.env.example`, e a trava de acesso do arquivo `SISTEMA_CLINICA.html`.

Método: revisão estática do código (leitura linha a linha das rotas e utilitários), mapeamento de todos os endpoints e seus decoradores de autorização, busca por padrões perigosos (concatenação de SQL, `eval`, `os.system`, path traversal), verificação do tratamento de segredos e comparação de `.env` com o modelo de exemplo.

Limitação: auditoria automatizada por agente de IA a partir de checklist. Não substitui pentest humano com terceiros (ver Seção 6, item 1).

3. Achados de severidade ALTA

A-01 · Chaves secretas em uso ainda são valores de exemplo (placeholder)

Severidade: ALTA

Onde: `.env` (SECRET_KEY, JWT_SECRET_KEY, DB_PASSWORD)

Evidência: o arquivo `.env` em uso é idêntico ao `.env.example`. As variáveis `SECRET_KEY` e `JWT_SECRET_KEY` têm 17 caracteres e correspondem ao padrão de placeholder do modelo — não são segredos fortes e aleatórios.

Risco: a `JWT_SECRET_KEY` é a chave usada para assinar os tokens de login. Se ela for um valor de exemplo (conhecido, curto ou previsível), um atacante pode **forjar um token válido e se passar por administrador** sem precisar de senha, obtendo acesso total a prontuários e ao financeiro. O mesmo vale para a senha do banco.

Correção: gerar segredos fortes e únicos antes de subir a aplicação, por exemplo com `python -c "import secrets; print(secrets.token_urlsafe(48))"` para `SECRET_KEY` e `JWT_SECRET_KEY` e uma senha aleatória longa para `DB_PASSWORD`. Nunca reutilizar os valores do `.env.example`. Confirmar que o `.env` real nunca é versionado (o `.gitignore` já o ignora — verificado).

A-02 · Senha de administrador do seed de demonstração é pública

Severidade: ALTA (se o seed rodar em produção) / Informativo (só em demo)

Onde: `database/seed_demo.sql`

Evidência: o script define o hash bcrypt do usuário `admin@clinica.local` correspondente à senha `TesteSenha123!`, documentada em texto no próprio arquivo.

Risco: qualquer pessoa com acesso ao repositório conhece a senha do admin. Se o `seed_demo.sql` for executado por engano em um ambiente real, o sistema nasce com uma conta de administrador de senha pública.

Correção: manter o `seed_demo.sql` estritamente para desenvolvimento/demonstração e **nunca** executá-lo em produção. Em produção, criar o primeiro admin com uma senha forte e única (o `schema.sql` já cria o admin com hash placeholder inválido, o que impede login acidental — bom). Trocar a senha de demonstração após o primeiro acesso, mesmo em ambiente de teste compartilhado.

4. Achados de severidade MÉDIA

M-01 · Dados pessoais do paciente legíveis por qualquer usuário logado

Onde: `GET /pacientes`, `GET /pacientes/<id>`, `GET /pacientes/<id>/enderecos`, `GET /agenda`

Evidência: essas rotas de leitura usam apenas `@jwt_required()`, sem `@require_perfil`. Diferente do prontuário (já restrito a perfil clínico), o cadastro do paciente — que inclui dados pessoais (CPF, contato, endereço) protegidos pela LGPD — pode ser lido por qualquer perfil autenticado.

Risco: exposição de dados pessoais entre perfis maior do que o necessário. Menos grave que o vazamento de prontuário (que era o risco crítico, já corrigido), mas ainda é uma superfície de exposição de PII.

Correção: avaliar se recepção/financeiro realmente precisam de todos os campos do paciente. Se sim, manter; se não, restringir os campos sensíveis (ex.: CPF, endereço) a perfis específicos ou criar uma versão reduzida do cadastro para perfis não clínicos.

M-02 · Proteção contra força bruta enfraquecida com múltiplos workers

Onde: `backend/Dockerfile` + `RATELIMIT_STORAGE_URI=memory://`

Evidência: o rate limit de login (5/min por IP+e-mail) usa armazenamento em memória (`memory://`), mas o Dockerfile inicia o Gunicorn com `--workers 4`. Cada um dos 4 processos mantém sua própria contagem, então na prática o limite efetivo é multiplicado (aprox. 4×) — enfraquecendo a proteção contra força bruta já na configuração atual, não só em um cenário futuro.

Risco: um atacante pode fazer mais tentativas de senha por minuto do que o limite pretendido.

Correção: apontar `RATELIMIT_STORAGE_URI` para um armazenamento compartilhado (ex.: `redis://...`), de modo que os 4 workers contem juntos. O `.env.example` já sugere isso; falta aplicar.

M-03 · Tokens JWT não podem ser revogados (logout apenas no cliente)

Onde: `backend/app/routes/auth.py`

Evidência: não há lista de bloqueio (blocklist) de tokens. O token de acesso vale 30 min e o de refresh vale 7 dias; um logout só descarta o token no navegador.

Risco: se um token for roubado (ex.: máquina comprometida), ele continua válido até expirar, sem forma de invalidá-lo no servidor. O refresh de 7 dias amplia essa janela.

Correção: implementar blocklist de `jti` (Flask-JWT-Extended tem suporte nativo, usando Redis) para permitir logout/revogação server-side, e considerar rotação do refresh token a cada uso.

M-04 · Aplicação não impõe HTTPS/HSTS por si só

Onde: configuração de implantação

Evidência: o Flask/Gunicorn serve HTTP na porta 5000; o projeto assume um proxy/TLS na frente, mas isso não é forçado pela aplicação.

Risco: sem HTTPS na frente, senhas e tokens trafegam sem criptografia e podem ser interceptados na rede.

Correção: garantir TLS terminado por proxy reverso (Nginx/Traefik/load balancer) em produção, com redirecionamento de HTTP para HTTPS e cabeçalho HSTS. Documentar isso como pré-requisito de deploy.

5. Achados de severidade BAIXA / informativos

B-01 · Sem autenticação em duas etapas (MFA) para administrador

O perfil administrador tem acesso amplo a dados clínicos e financeiros. Recomenda-se MFA para essas contas antes do uso real.

B-02 · Política de senha limitada ao mínimo de 8 caracteres

A criação de usuário exige só 8 caracteres, sem regra de complexidade nem checagem contra senhas comuns. Considerar reforço para contas privilegiadas.

B-03 · Sem controle de vínculo profissional-paciente na leitura clínica

Já registrado na revisão anterior: qualquer dentista pode ler o prontuário de qualquer paciente. Aceitável operacionalmente, mas rever se a política da clínica exigir restrição ao dentista responsável.

B-04 · A trava de senha do arquivo SISTEMA_CLINICA.html não é segurança real

A tela de senha do arquivo HTML avulso roda inteiramente no navegador (hash simples embutido no próprio arquivo). Serve apenas como barreira de acesso casual e pode ser contornada por quem inspecionar o arquivo. A segurança de verdade é a do backend descrito neste relatório. Não confundir os dois: o HTML avulso é uma demonstração de interface, não o sistema em produção.

6. O que já está correto (verificado)

Controle	Situação
Autorização em todas as rotas de escrita (login + perfil)	OK
Leitura de prontuário/odontograma restrita a perfil clínico (correção anterior confirmada)	OK
Senhas com bcrypt; `senha_hash` nunca serializado nas respostas	OK
	OK

Consultas SQL parametrizadas (`db.text` com `:param`); sem concatenação — nenhum SQL injection encontrado	
Upload de exames: whitelist de extensão, limite de 10 MB, checagem de magic bytes, nome UUID, `secure_filename`	OK
CORS restrito a `FRONTEND_URL` , nunca `"*"`	OK
Segredos via variável de ambiente; app recusa iniciar sem `SECRET_KEY` / `JWT_SECRET_KEY`	OK
`.gitignore` ignora `.env` (segredos fora do versionamento)	OK
JWT no cabeçalho `Authorization` (não em cookie) → CSRF não se aplica	OK
Container roda como usuário não-root (`appuser`)	OK
Rate limit no login (5/min por IP+e-mail; 20/min por IP)	OK (ver M-02)
Mensagem de login genérica ("Credenciais inválidas") — sem enumeração de usuários	OK
Auditoria (`log_auditoria`) em operações de escrita sensíveis	OK
`FLASK_DEBUG=false` por padrão; erro 500 não expõe stack trace	OK

7. Recomendações antes de uso com pacientes reais

- **Resolver A-01 e A-02** (segredos fortes; nunca rodar o seed de demonstração em produção).
- **Pentest humano** por terceiros — esta auditoria por IA não o substitui.
- **HTTPS/TLS** obrigatório na frente da aplicação, com HSTS.
- **Redis** para rate limiting compartilhado entre os 4 workers (M-02) e para blocklist de JWT (M-03).
- **MFA** para contas de administrador.
- **Backups testados** de verdade (restauração periódica, não só agendamento).
- **Adequação formal à LGPD**: política de privacidade, termo de consentimento, tempo de retenção, processo de exclusão/portabilidade de dados.
- **Monitoramento** e alertas para volume anormal de logins malsucedidos.

8. Conclusão

O backend do MedFlow apresenta uma **arquitetura de segurança sólida no nível da aplicação** — os riscos clássicos (SQL injection, senhas em texto puro, rotas de escrita desprotegidas, upload malicioso, CORS aberto) foram tratados ou não estão presentes. Os dois pontos de maior atenção são de **configuração de implantação**, não de código: substituir os segredos de exemplo por valores reais e fortes, e garantir que a senha pública do seed de demonstração jamais chegue a produção.

Com os achados Alta resolvidos, HTTPS na frente e um pentest humano, o sistema atinge um patamar adequado para operar com dados reais de pacientes, sempre acompanhado da devida adequação à LGPD.

Relatório gerado em 30/09/2026 · Auditoria de código e banco de dados · Documento confidencial destinado ao responsável do projeto. As correções de configuração citadas devem ser aplicadas no ambiente de implantação, fora do repositório de código.